

Multi – Level linked list
–Insert After Element(For Child Insertion)
–Program Analysis

```
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}

void insertAfterElement()
{
    int target, data, type;
    cout << "Enter the element to search for: ";
    cin >> target;

    // Simple search (Only searches main level for
    // simplicity in this demo)
    Node *temp = head;
    while (temp != nullptr && temp->data != target)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Element not found in main list." << endl;
        return;
    }
}
```

```

    cout << "Enter data to insert: ";
    cin >> data;
    Node *newNode = createNode(data);

    cout << "Insert as (1) Next Node or (2) Child Node? ";
    cin >> type;

    if (type == 1)
    {
        // Standard Linked List Insertion
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted " << data << " as NEXT of " <<
target << endl;
    }
    else if (type == 2)
    {
        // Multilevel Insertion
        // If a child already exists, we link the new node
to the front of the child list
        newNode->next = temp->child;
        temp->child = newNode;
        cout << "Inserted " << data << " as CHILD of " <<
target << endl;
    }
    else
    {
        cout << "Invalid option." << endl;
        free(newNode);
    }
}
.....
int main()
{
    int choice;
    while (true)
    {

```

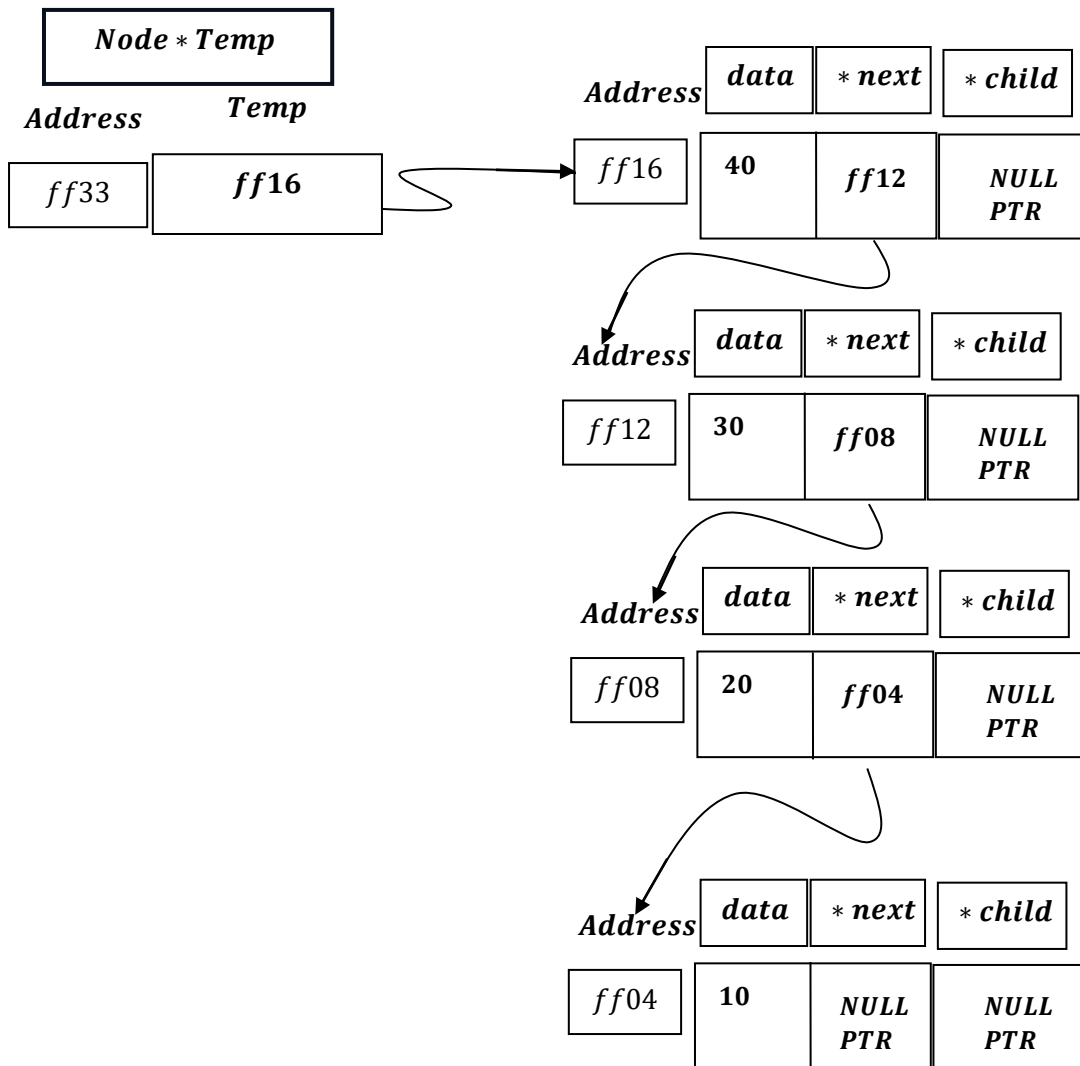
```
        cout << "4. Insert After Element (Link or Child)" << endl;
        cin >> choice;

        switch (choice)
        {
        case 4:
            insertAfterElement();
            break;
        default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

Program Analysis

Lets say, we have:

Child List Insertion

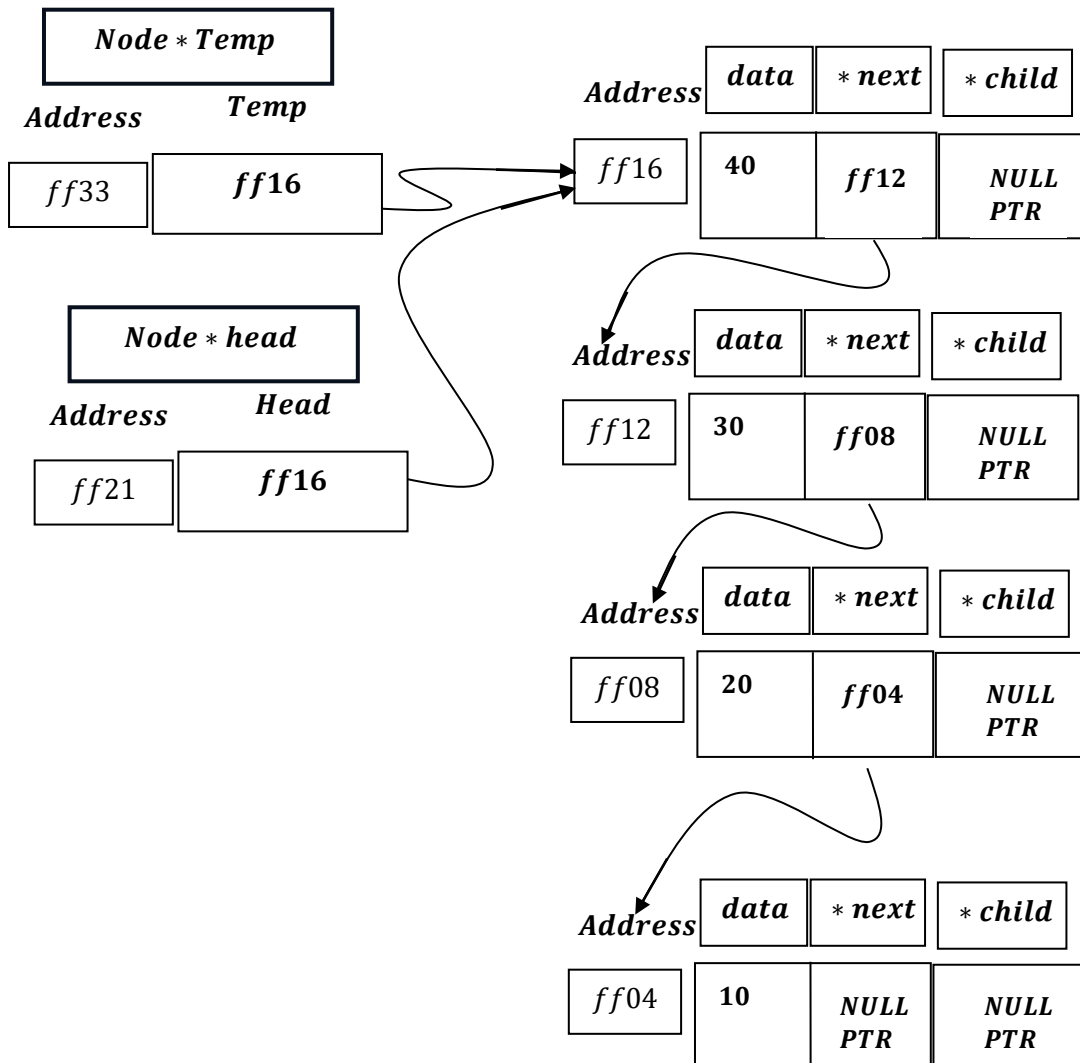


```
int target;  
cout << "Enter the element to search for: ";  
cin >> target;
```

Say target: 20

```
Node *temp = head;
```

TEMP = HEAD = ff16



```
while (temp != nullptr && temp->data != target)
{
    temp = temp->next;
}
```

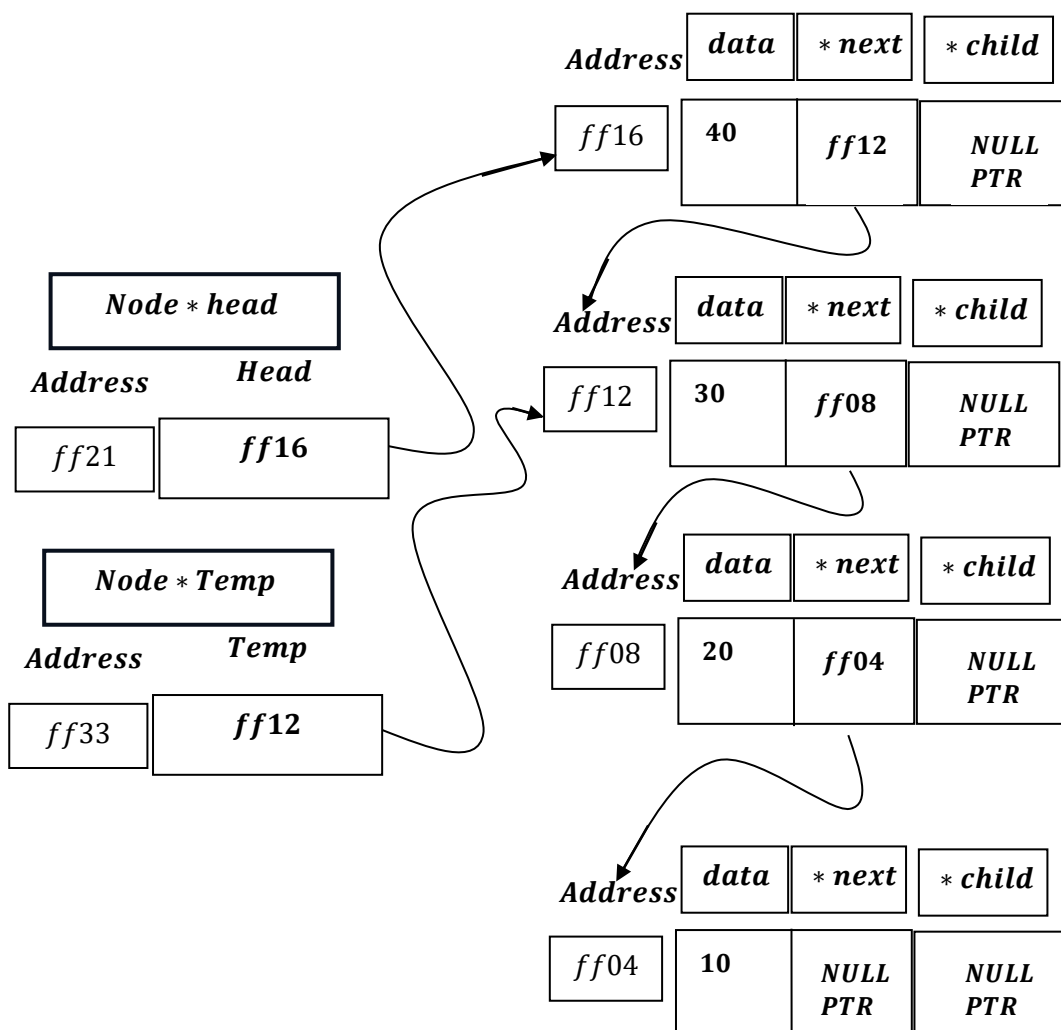
Here , $temp \rightarrow data \neq Target: 20$ $\left[\begin{array}{c} \text{Will move} \\ \text{to previous node} \\ \text{of Target} \end{array} \right]$ and

$temp \neq NULLPTR$ $\left[\begin{array}{c} \text{Not} \\ \text{Out of Bound} \end{array} \right]$:

[Loop]

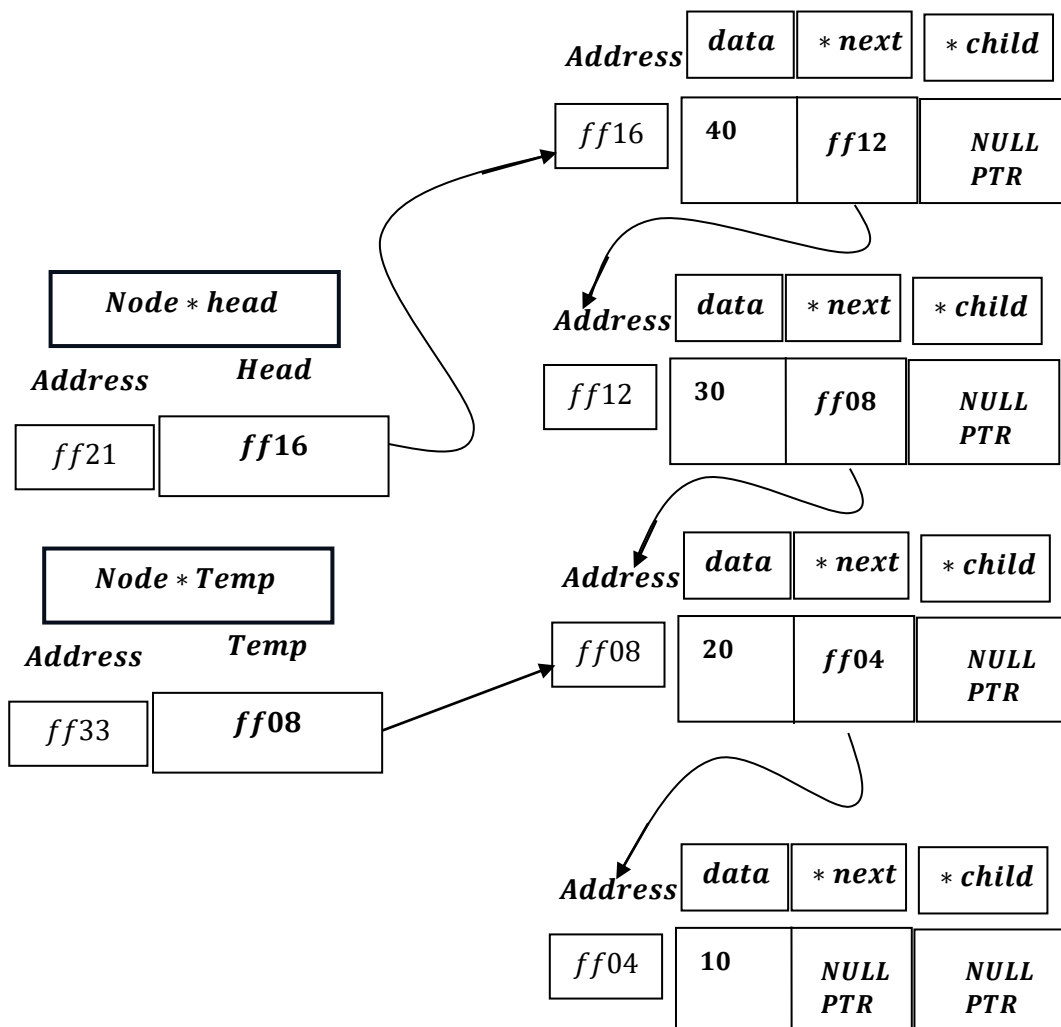
When $Temp \rightarrow Data : 40 \neq 20$ and $Temp \neq NULLPTR$: (Condition is True)

$TEMP = TEMP \rightarrow NEXT = ff12$



When $Temp \rightarrow Data : 30 \neq 20$ and $Temp \neq NULLPTR$: (Condition is True)

$TEMP = TEMP \rightarrow NEXT = ff08$



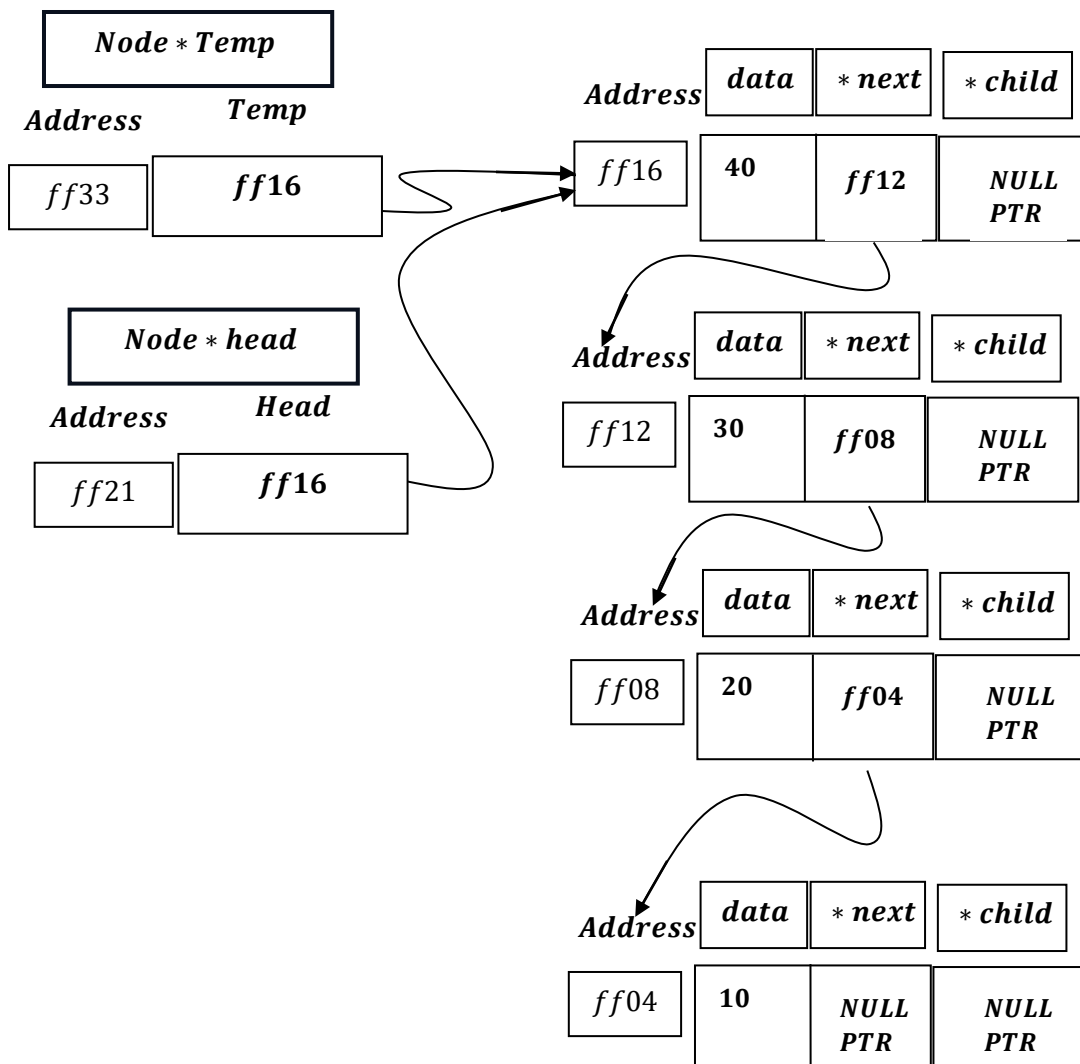
*When Temp → Data : 20 ≠ 20 and Temp
≠ NULLPTR: (Condition is False)*

Loop Exits.

When Element Not Found

```
if (temp == nullptr)
{
    cout << "Element not found in main list." << endl;
    return;
}
```

Say target is: 05.



```
while (temp != nullptr && temp->data != target)
{
    temp = temp->next;
}
```

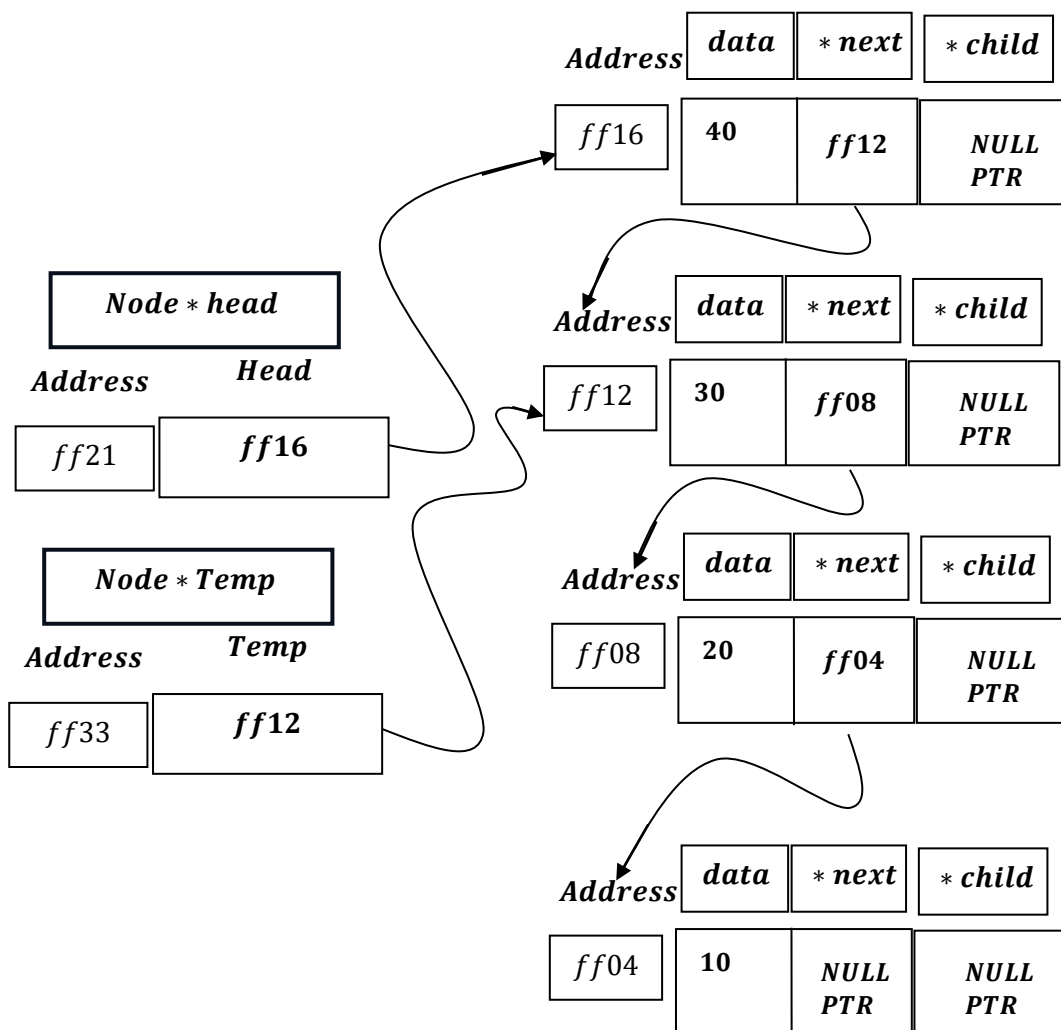
Here , $temp \rightarrow data \neq Target: 5$ $\left[\begin{array}{c} \text{Will move} \\ \text{to previous node} \\ \text{of Target} \end{array} \right]$ and

$temp \neq NULLPTR$:

[Loop]

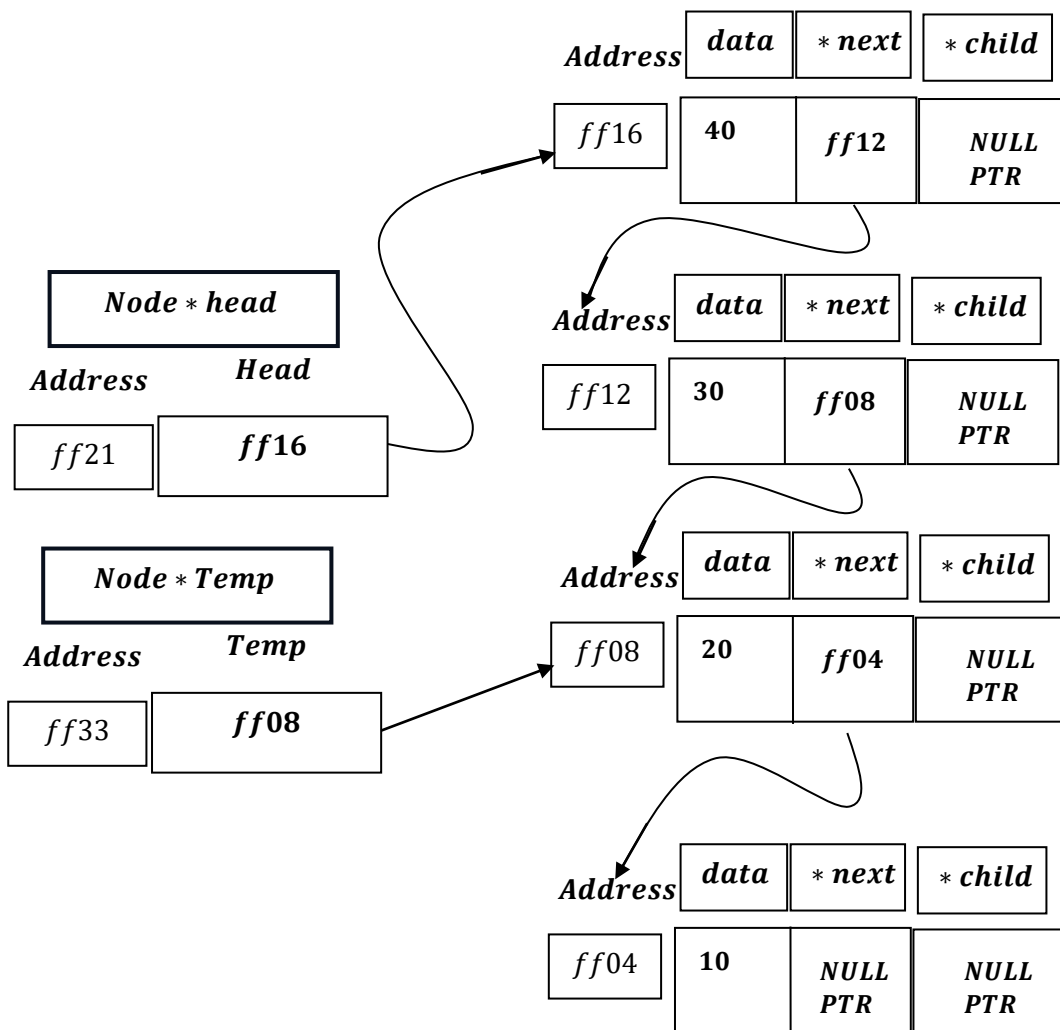
When $Temp \rightarrow Data : 40 \neq 5$ and $Temp \neq NULLPTR$: (Condition is True)

$TEMP = TEMP \rightarrow NEXT = ff12$



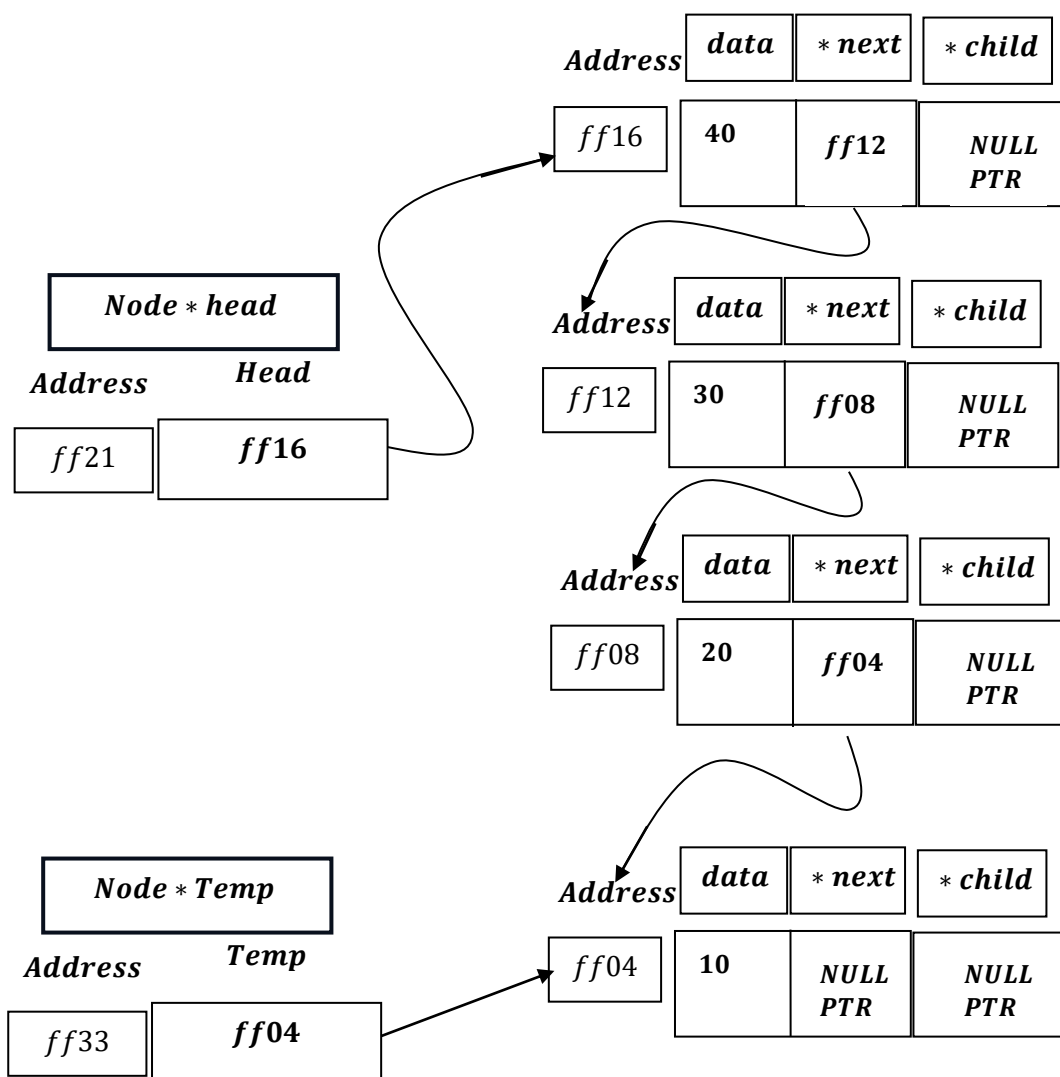
When $Temp \rightarrow Data : 30 \neq 5$ and $Temp \neq NULLPTR$: (Condition is True)

$TEMP = TEMP \rightarrow NEXT = ff08$



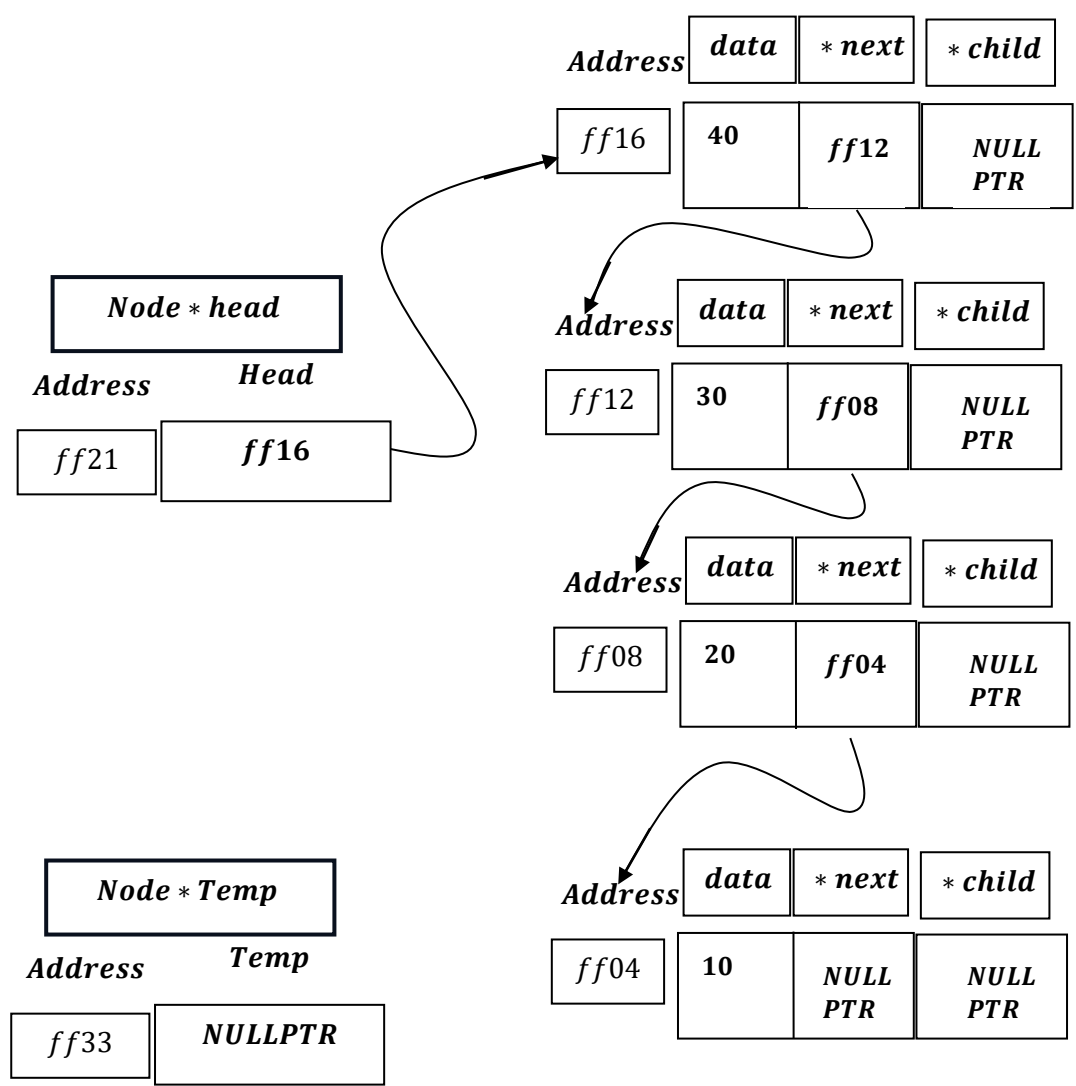
When $Temp \rightarrow Data : 20 \neq 5$ and $Temp \neq NULLPTR$: (Condition is True)

$TEMP = TEMP \rightarrow NEXT = ff04$



*When Temp → Data : 10 ≠ 5 and Temp
≠ NULLPTR: (Condition is True)*

TEMP = TEMP → NEXT = NULLPTR



As Temp = NULLPTR , Condition fails and loop exits.

```

if (temp == nullptr)
{
    cout << "Element not found in main list." << endl;
    return;
}

```

As ,Temp = NULL :

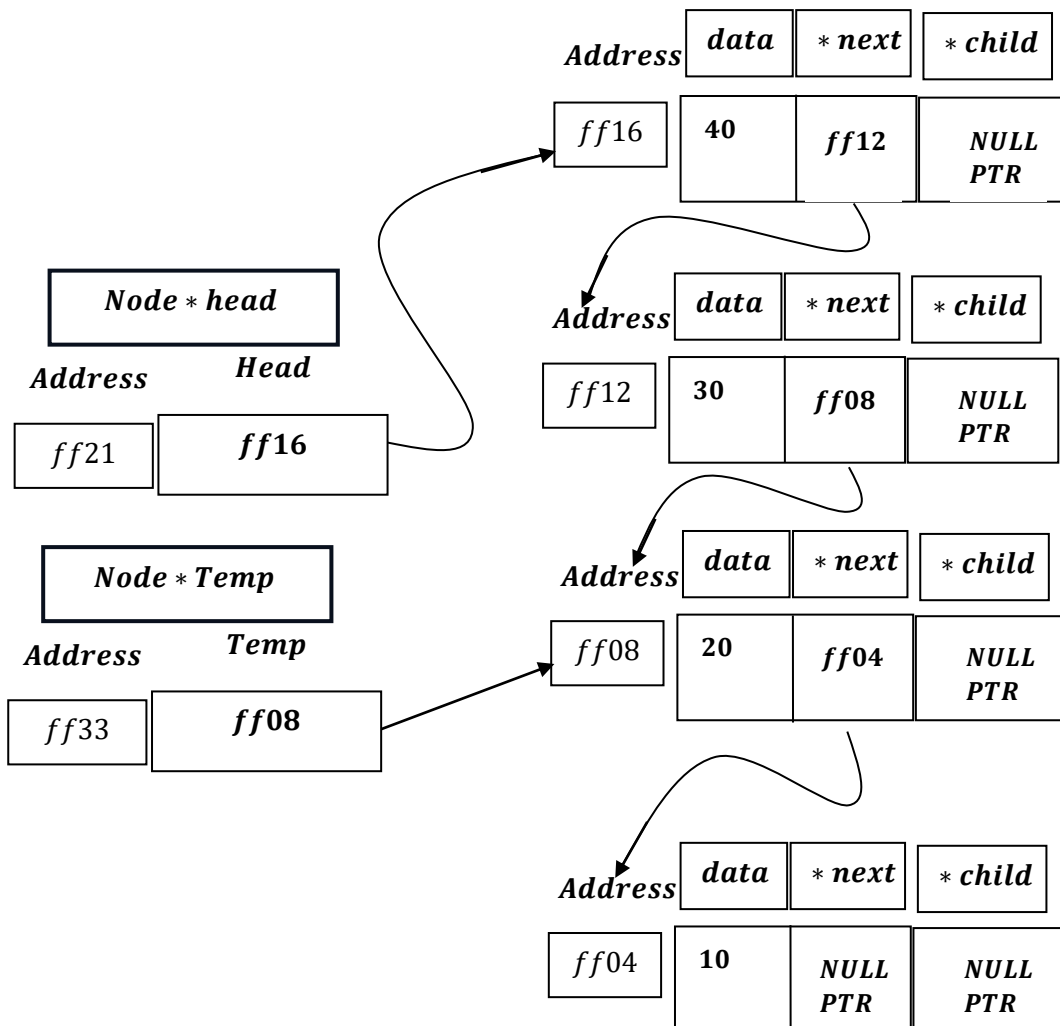
It will print : `` Element not found in main list``.

Return will return void and exit the function.

Note first we need to find out the element in the main list.

Now, lets return when $TEMP \neq NULLPTR$ $\left[\begin{array}{c} \text{Not Out} \\ \text{of} \\ \text{Bound} \end{array} \right]$

Target: 20



```
cout << "Enter data to insert: ";
cin >> data;
```

lets say, We want to enter data : 15

```
Node *newNode = createNode(data);
```

```
Node * newNode = createNode(data: 15);
```

Next,

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64-bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

→ data(an integer) is typically 4 bytes.

→ next(a pointer pointing to another Node) is typically 8 bytes.

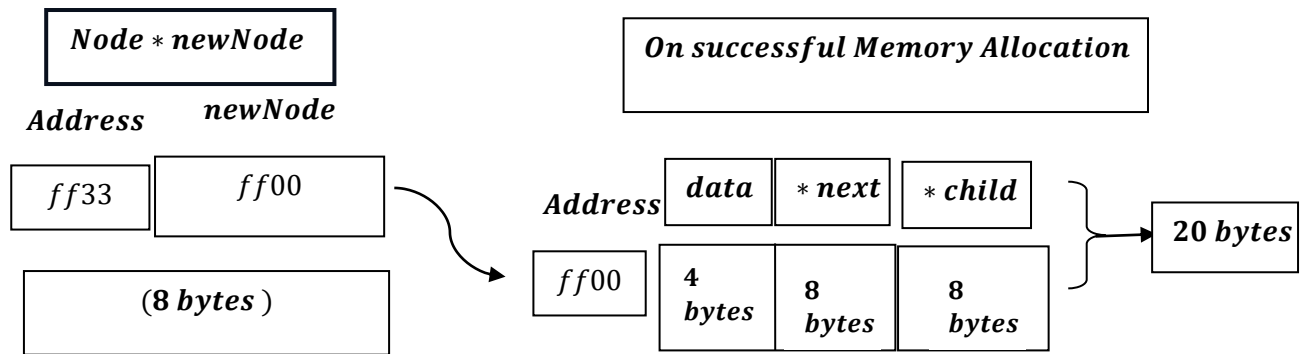
→ child(a pointer pointing to child node) is typically 8 bytes.

→ Therefore, the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for child).

What does it mean?

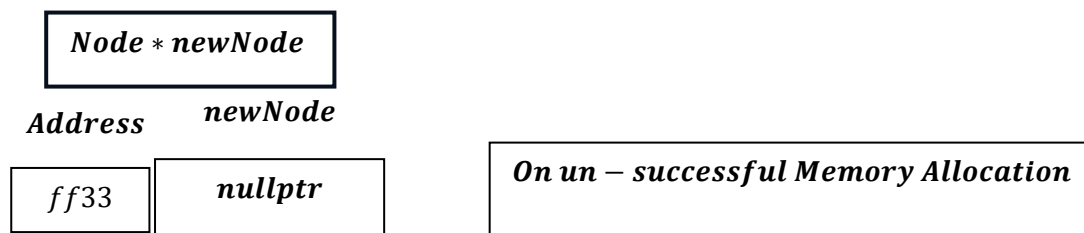
→ newNode will try to allocate memory for integer data for 4 bytes and next pointer variable i. e. typically 8 bytes, also child next pointer variable also typically 8 bytes hence assumed total space consumed is total (4 + 8 + 8)bytes = 20 bytes.

Say,



But on Un – Successful memory allocation:

→ if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer.



Now, if the value of the pointer variable newNode is nullptr, then:

One may output : `Overflow or Memory Allocation Failed as a message.`

And our data input is : 15.

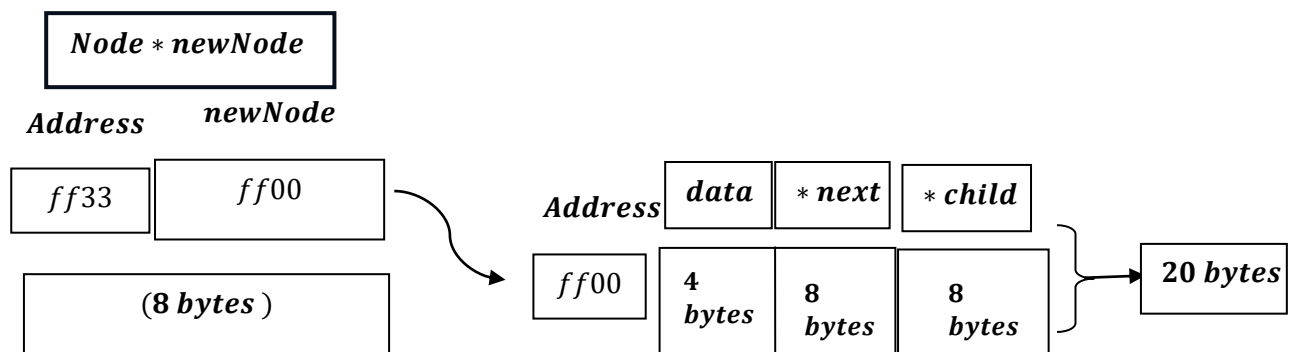
```
Node *createNode(int data){...}
```

Return type is struct type pointer variable as mentioned here:

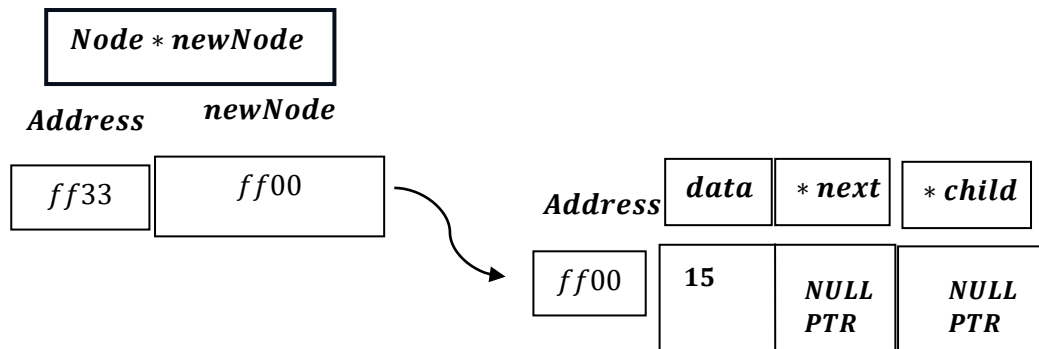
*Node * and function name is createNode(data : 15).*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

We got:



```
newNode->data = data;  
newNode->next = nullptr;  
newNode->child = nullptr;
```



```
return newNode;
```

*As we already know , Return type is struct type pointer variable .
Return `newNode` will return the value stored inside `newNode`,
pointer :*

i. e. `return newNode : ff00` .

Now,

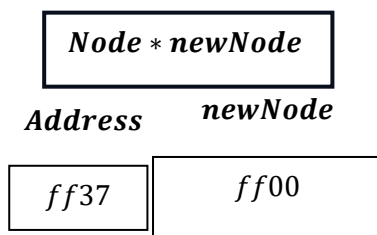
```
Node *newNode = createNode(data);
```

`Node * newNode = ff00.`

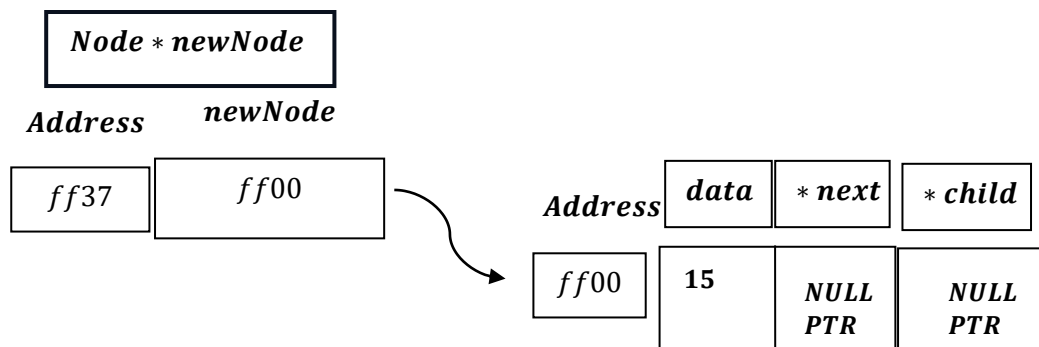
This newNode is another pointer variable ,under scope of insertAfterElement() function,not to be mixed with createNode() function .

*For * newNode of insertAfterElement() function:*

*Node * newNode = ff00.*



*Now pointer * newNode connecting to address ff00.*



```
cout << "Insert as (1) Next Node or (2) Child Node? ";
cin >> type;
```

Next Node is entering newNode in the child list .

Therefore ,User Input ,Type: 2 .

```

if (type == 1)
{...    }
else if (type == 2)
{
    // Multilevel Insertion
    // If a child already exists, we link the new node
to the front of the child list
    newNode->next = temp->child;
    temp->child = newNode;
    cout << "Inserted " << data << " as CHILD of " <<
target << endl;
}

```

```

newNode->next = temp->child;

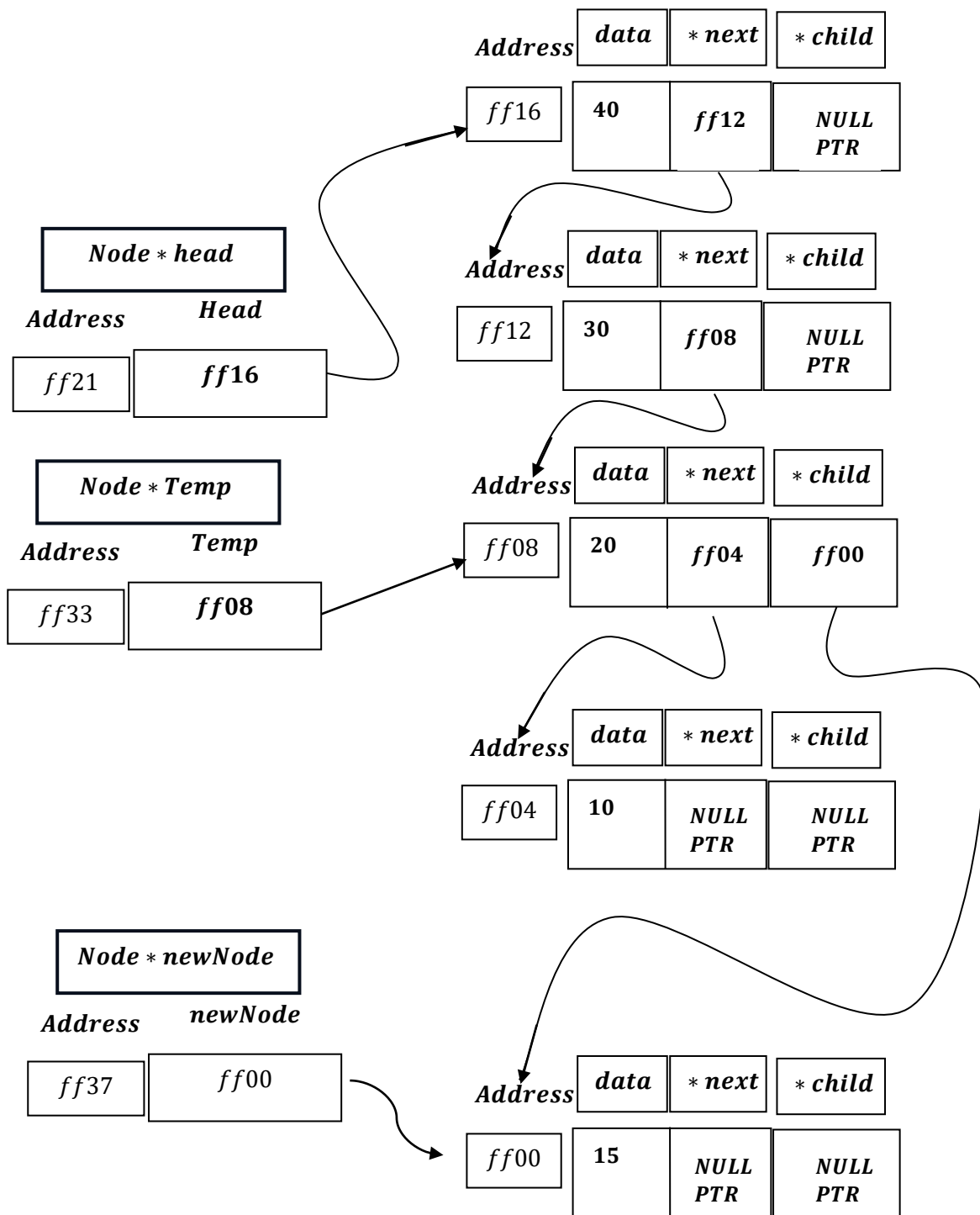
```

$newNode \rightarrow next = temp \rightarrow child = NULLPTR$
[
*As it is first time
insertion, hence
no change.*
]

```

temp->child = newNode;
temp → child = newNode = ff00.

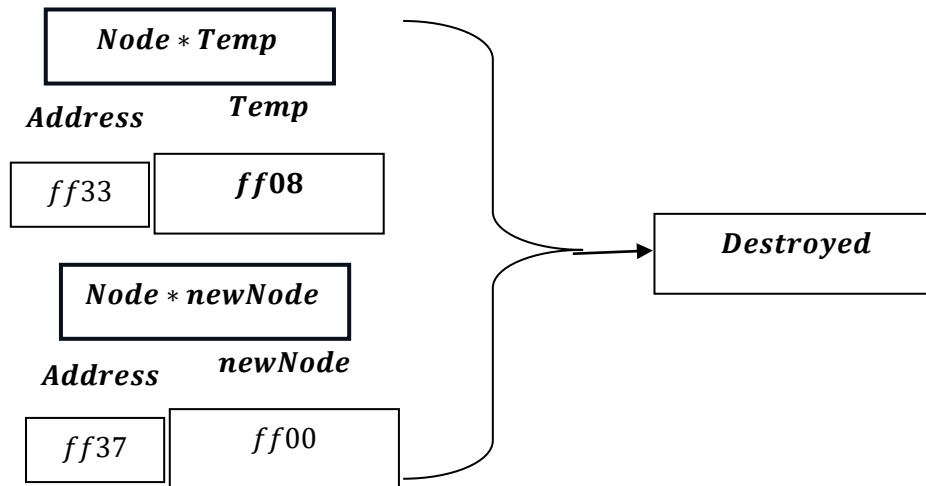
```



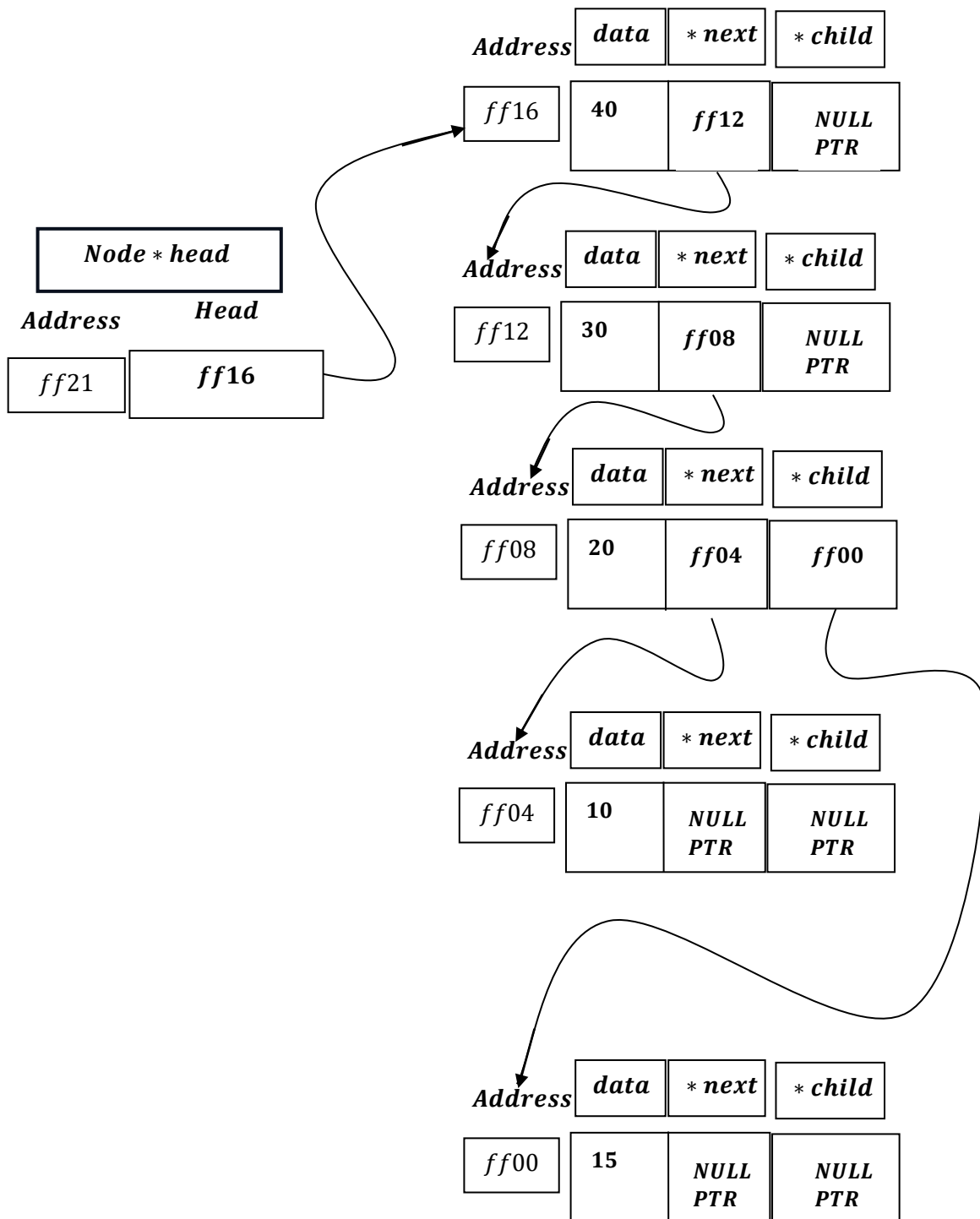
```
cout << "Inserted " << data << " as CHILD of " <<
target << endl;
```

It will print: ``Inserted``, data: 15 , ``as CHILD of``, target: 20.

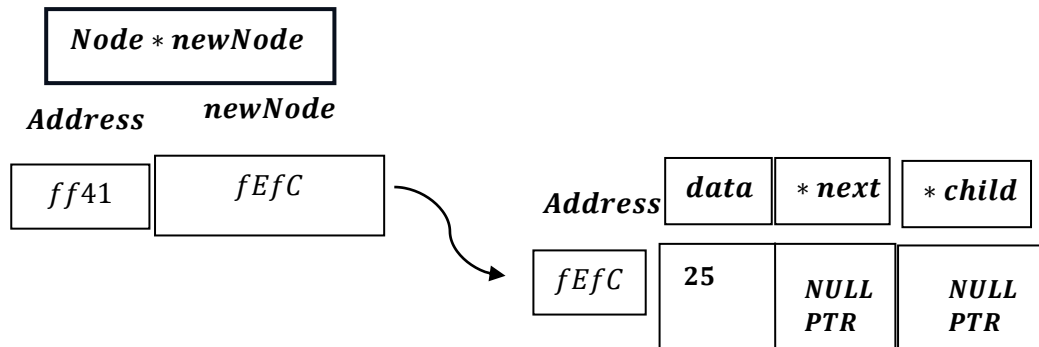
As ,InsertAfterElement() ,function ends :



And we are left with:



Now ,say again our target is 20 and ,we want to enter in the child of target node ,whose data: 20. And data to be entered is : 25.



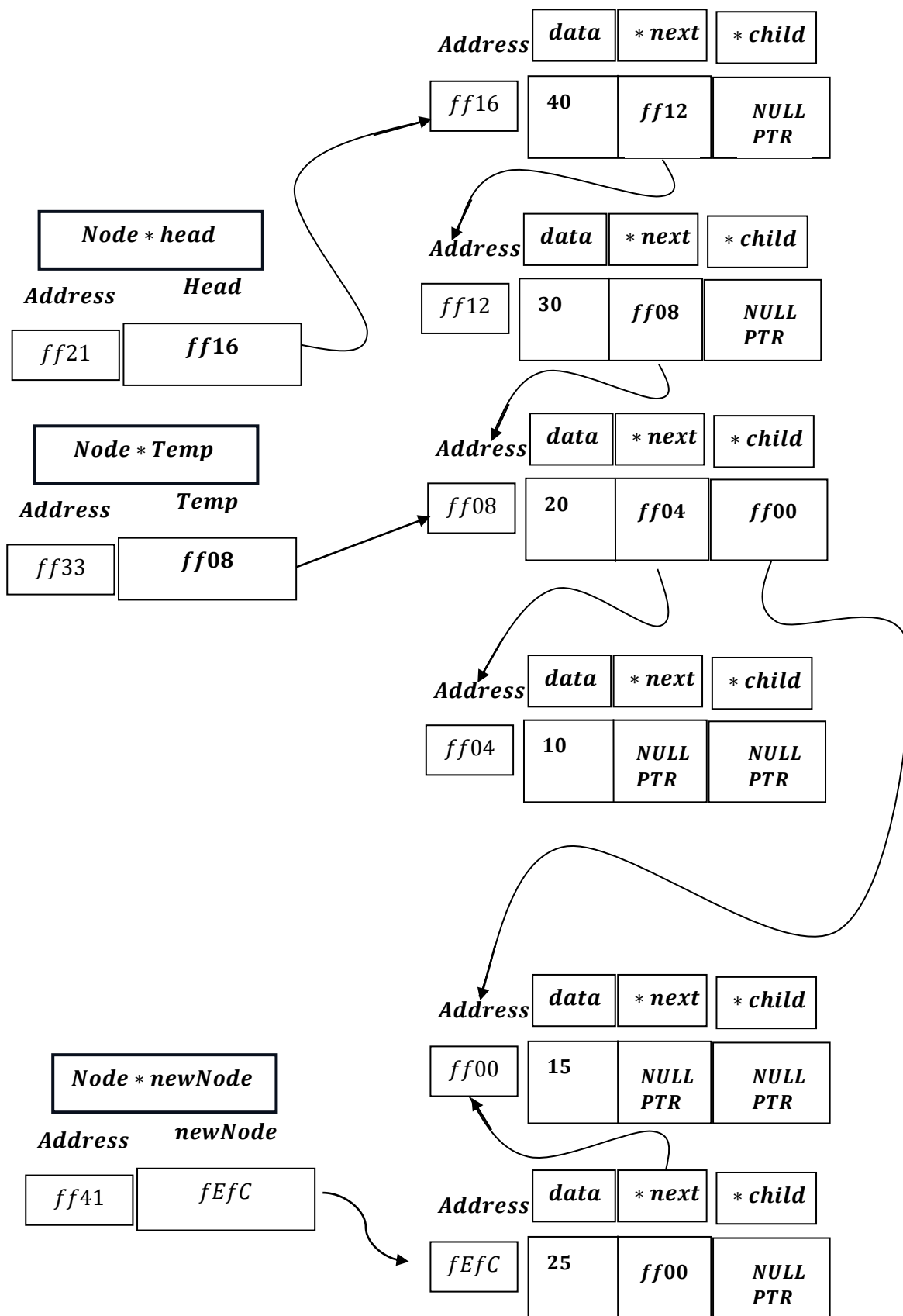
```

if (type == 1)
{...}
else if (type == 2)
{
    // Multilevel Insertion
    // If a child already exists, we link the new node
to the front of the child list
    newNode->next = temp->child;
    temp->child = newNode;
    cout << "Inserted " << data << " as CHILD of " <<
target << endl;
}

```

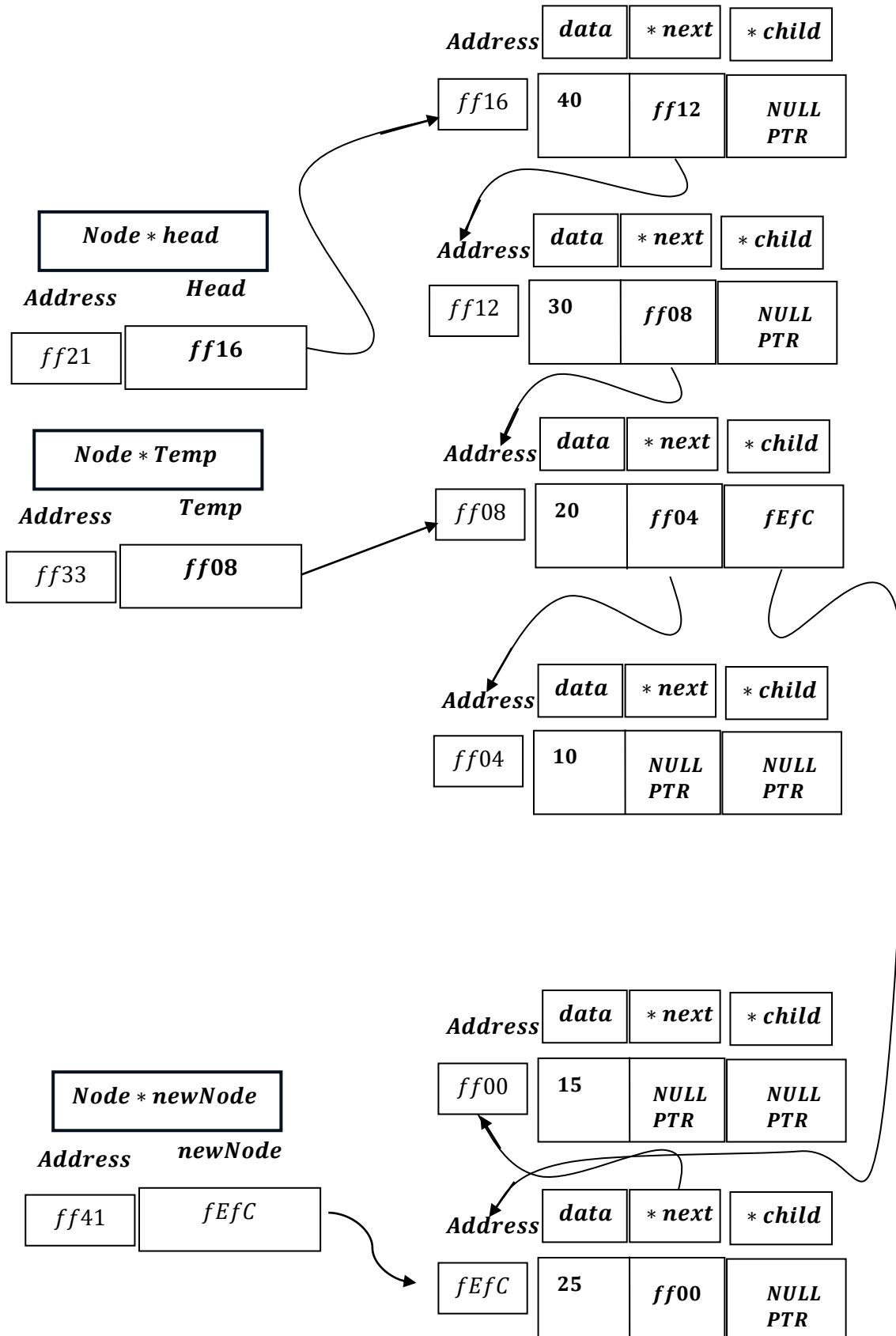
```
newNode->next = temp->child;
```

newNode → *next* = *temp* → *child* = *ff00*.

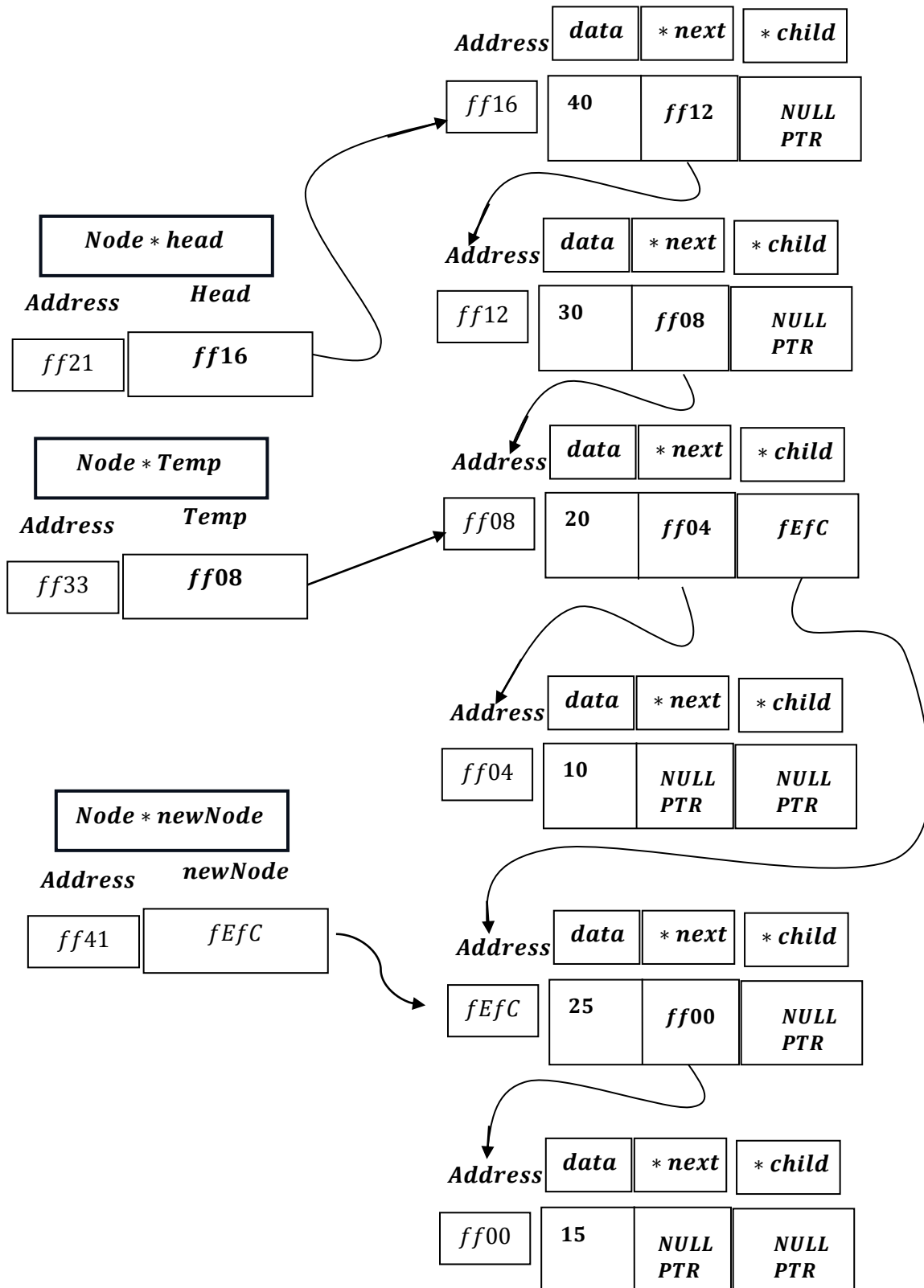


```
temp->child = newNode;
```

$Temp \rightarrow child = newNode = fEfC$

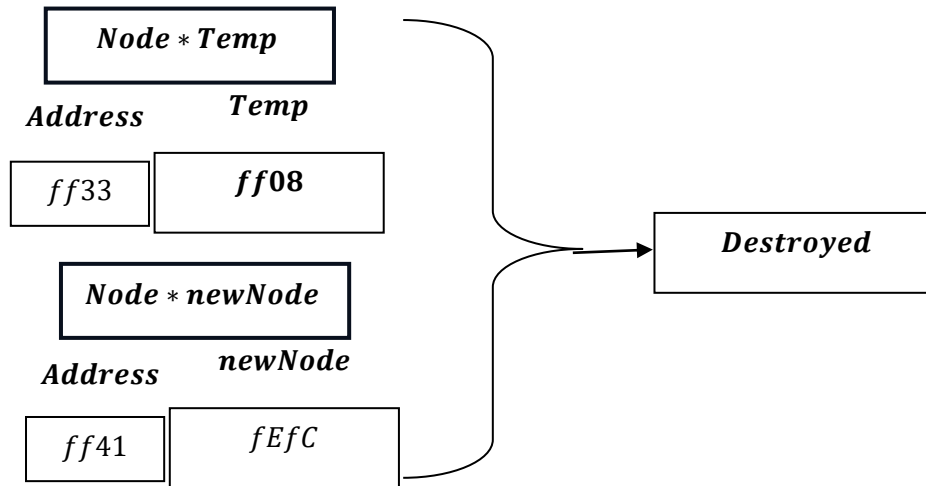


Actually it is:

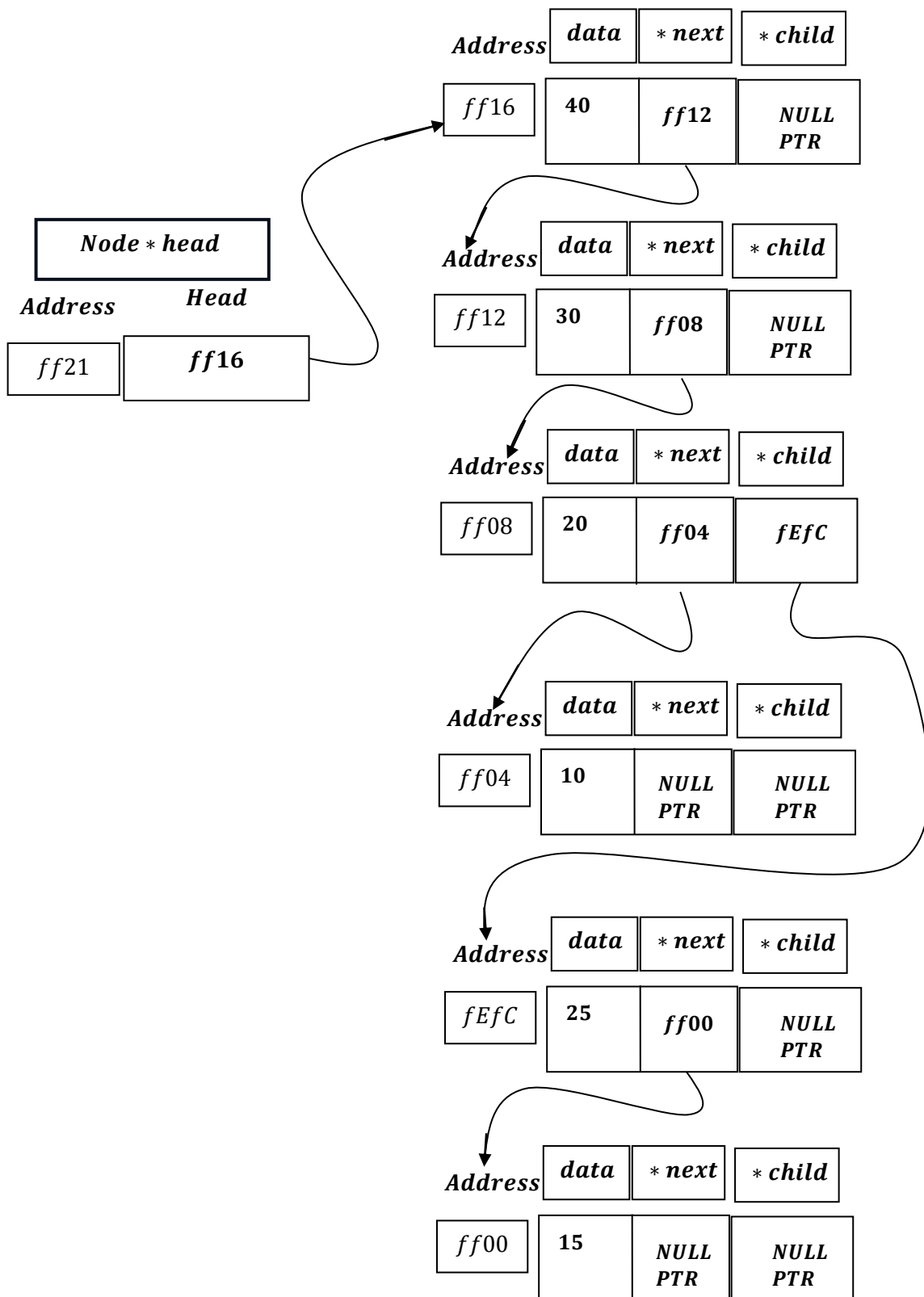


That is if a child already exists ,we link the newNode infront of the child list.

As ,InsertAfterElement() ,function ends :



And we are left with:



```

else
{
    cout << "Invalid option." << endl;
    free(newNode);
}

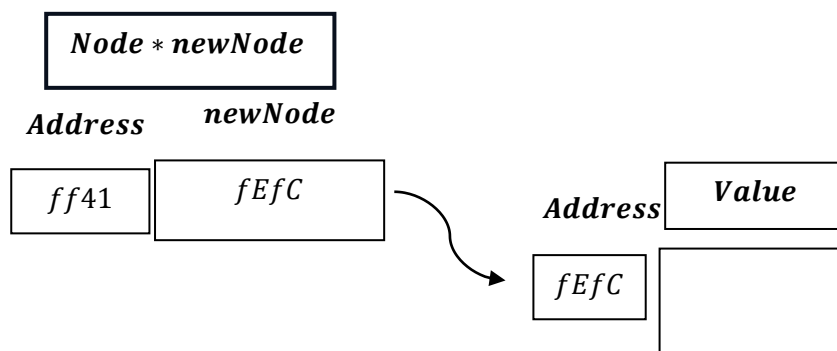
```

If not attached to MAIN LIST nor the CHILD , then:

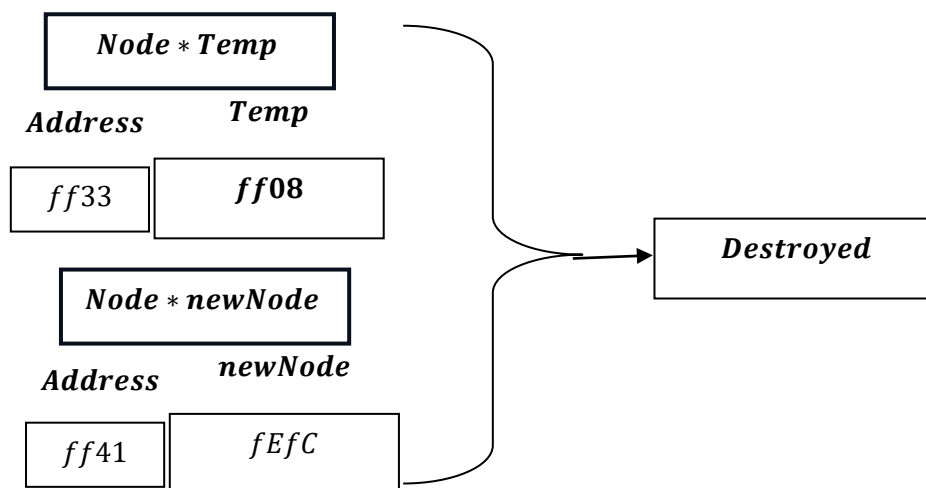
Print: ``Invalid Option.``

Free(newNode)

This will:



As ,InsertAfterElement() ,function ends :



And we are left with:

